

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Redes

Semestre II – 2026



Excelencia que trasciende

DELVALLE
GRUPO EDUCATIVO

Laboratorio 2

Esquemas de detección y corrección de errores

Integrantes:

Nájera Marakovits, Ingrid Nina Alessandra – 231088

Ramírez Velásquez, Diego Alejandro – 23601

Índice

1. Descripción de la práctica	2
2. Objetivos	2
3. Implementación	2
3.1. Emisor en Python	2
3.2. Receptor en Java	3
3.3. Flujo utilizado	3
4. Algoritmos utilizados	3
4.1. Código de Hamming	3
4.2. Fletcher checksum	4
5. Metodología de pruebas	4
5.1. Resumen de las pruebas	4
6. Resultados	6
6.1. Pruebas sin errores	6
6.2. Pruebas con un error	19
6.3. Pruebas con dos errores	22
7. Análisis gráfico	24
7.1. Cantidad de información adicional	24
7.2. Resultados correctos según la función del algoritmo	25
8. Discusión	26
8.1. Funcionamiento de Hamming	26
8.2. Funcionamiento de Fletcher-16	26
8.3. Comparación	27
9. Comentario grupal	27
10. Conclusiones	27
11. Repositorio	28

1. Descripción de la práctica

En este laboratorio se desarrollaron dos programas para simular la transmisión de información por un medio que puede introducir errores. El objetivo principal fue comprobar cómo funcionan los mecanismos de detección y corrección de errores utilizados en la capa de enlace.

El programa emisor fue desarrollado en Python y el receptor fue desarrollado en Java. En esta fase, la comunicación se realizó de forma manual: la trama producida por el emisor se copió y se ingresó en el receptor.

La información enviada utilizó el siguiente formato:

ALGORITMO | TRAMA | LONGITUD | PADDING | TIPO

Por ejemplo:

HAMMING | 0110011 | 4 | 0 | BINARIO

El receptor separa estos campos, verifica la integridad de la trama y muestra si el mensaje llegó correctamente, si fue corregido o si debe ser descartado.

2. Objetivos

- Implementar un algoritmo de corrección de errores.
- Implementar un algoritmo de detección de errores.
- Probar mensajes de diferentes longitudes.
- Observar el comportamiento de los algoritmos ante uno o más errores.
- Comparar la cantidad de información adicional utilizada por cada algoritmo.

3. Implementación

3.1. Emisor en Python

El emisor recibe una cadena binaria o un texto ASCII. Cuando se ingresa texto, cada carácter se convierte a una representación binaria de ocho bits.

Después, el usuario selecciona el algoritmo que desea utilizar. El programa genera la trama con los bits adicionales y muestra una línea que puede copiarse directamente en el receptor.

El emisor también permite aplicar ruido automáticamente con una probabilidad determinada. Para las pruebas de este informe, los bits se modificaron manualmente para controlar exactamente la cantidad de errores agregados.

3.2. Receptor en Java

El receptor recibe la línea completa generada por el emisor y separa los datos utilizando el carácter |.

Posteriormente, selecciona el proceso correspondiente:

- Para Hamming, calcula el síndrome y corrige el bit indicado.
- Para Fletcher, vuelve a calcular el checksum y lo compara con el recibido.

El receptor también recupera el mensaje binario original. Cuando el tipo de mensaje es ASCII, convierte los grupos de ocho bits nuevamente a caracteres.

3.3. Flujo utilizado

Mensaje original → Emisor en Python → Trama con redundancia → Copia manual → Receptor en Java → Resultado

4. Algoritmos utilizados

4.1. Código de Hamming

El código de Hamming se utilizó como algoritmo de corrección. Los bits de paridad se colocaron en posiciones que son potencias de dos:

$$1, 2, 4, 8, 16, \dots$$

La cantidad de bits de paridad se determinó mediante la condición:

$$2^r \geq m + r + 1$$

donde m es la cantidad de bits del mensaje y r es la cantidad de bits de paridad.

Se utilizó paridad par y las posiciones se contaron desde la izquierda. Cuando el receptor encuentra un síndrome distinto de cero, el valor del síndrome indica la posición del bit que debe invertirse.

Esta implementación puede corregir un error. Sin embargo, no incluye un bit de paridad global, por lo que no puede manejar de manera confiable dos o más errores.

4.2. Fletcher checksum

Fletcher se utilizó como algoritmo de detección de errores. Aunque el emisor permite Fletcher-8, Fletcher-16 y Fletcher-32, para las pruebas del informe se utilizó Fletcher-16.

Fletcher-16 divide los datos en palabras de ocho bits y utiliza el módulo 255. Para cada palabra D_i , se realizan los cálculos:

$$S_1 = (S_1 + D_i) \text{ mód } 255$$

$$S_2 = (S_2 + S_1) \text{ mód } 255$$

El checksum final está formado por S_2 seguido de S_1 . El receptor vuelve a calcular estos valores. Si el checksum calculado no coincide con el checksum recibido, la trama se descarta.

Fletcher puede detectar alteraciones, pero no indica la posición del error ni puede corregirlo.

5. Metodología de pruebas

Se utilizaron tres mensajes de diferentes longitudes:

- Mensaje corto: 1011, con 4 bits.
- Mensaje mediano: 11010110, con 8 bits.
- Mensaje largo: Hola, convertido a 32 bits ASCII.

Cada mensaje se procesó con Hamming y Fletcher-16. Se realizaron tres escenarios:

1. Trama sin errores.
2. Trama con un bit modificado.
3. Trama con dos bits modificados.

En total se realizaron 18 pruebas.

5.1. Resumen de las pruebas

No.	Mensaje	Algoritmo	Errores	Resultado
1	1011	Hamming	0	Trama válida y mensaje recuperado correctamente.
2	11010110	Hamming	0	Trama válida y mensaje recuperado correctamente.
3	Hola	Hamming	0	Trama válida y texto ASCII recuperado correctamente.
4	1011	Fletcher-16	0	Los checksums coincidieron y la trama fue aceptada.
5	11010110	Fletcher-16	0	Los checksums coincidieron y la trama fue aceptada.
6	Hola	Fletcher-16	0	Los checksums coincidieron y se recuperó el texto.
7	1011	Hamming	1	Se detectó y corrigió el error en la posición 4.
8	11010110	Hamming	1	Se detectó y corrigió el error en la posición 4.
9	Hola	Hamming	1	Se corrigió el error y se recuperó Hola.
10	1011	Fletcher-16	1	El checksum fue diferente y la trama fue descartada.
11	11010110	Fletcher-16	1	El checksum fue diferente y la trama fue descartada.
12	Hola	Fletcher-16	1	El checksum fue diferente y la trama fue descartada.
13	1011	Hamming	2	Se realizó una corrección incorrecta y se recuperó 1001.
14	11010110	Hamming	2	Se realizó una corrección incorrecta y se recuperó 11110110.
15	Hola	Hamming	2	Se realizó una corrección incorrecta y se recuperó hola.
16	1011	Fletcher-16	2	Se detectaron cambios y la trama fue descartada.
17	11010110	Fletcher-16	2	Se detectaron cambios y la trama fue descartada.
18	Hola	Fletcher-16	2	Se detectaron cambios y la trama fue descartada.

6. Resultados

6.1. Pruebas sin errores

En las primeras seis pruebas, las tramas se trasladaron del emisor al receptor sin modificar ningún bit.

Salida del emisor

```
PS C:\Users\ninan\OneDrive\Documentos\Escritorio\UVG\VIII SEMESTRE\Redes\Redes> & C:\V
neDrive/Documentos/Escritorio/UVG/VIII SEMESTRE/Redes/Redes/emisor.py"
=====
EMISOR - DETECCIÓN Y CORRECCIÓN DE ERRORES
=====

Tipo de mensaje:
1. Cadena binaria
2. Texto ASCII
Seleccione una opción: 1
Ingrese el mensaje binario: 1011

Algoritmo:
1. Hamming, corrección de un error
2. Fletcher checksum, detección de errores
Seleccione una opción: 1

Bits de paridad agregados: 3

Aplicación de ruido:
1. No aplicar ruido y modificar los bits manualmente
2. Aplicar ruido automáticamente según una probabilidad
Seleccione una opción: 1

=====
RESULTADO DEL EMISOR
=====

Mensaje original: 1011
Datos binarios originales: 1011
Tipo de mensaje: BINARIO
Algoritmo: HAMMING
Longitud original en bits: 4
Padding agregado: 0
Trama antes del ruido: 0110011
Probabilidad utilizada: 0.0
Trama que se enviará: 0110011
Bits modificados por el ruido: ninguno

COPIAR Y ENVIAR AL RECEPTOR:
HAMMING|0110011|4|0|BINARIO
=====
PS C:\Users\ninan\OneDrive\Documentos\Escritorio\UVG\VIII SEMESTRE\Redes\Redes> |
```

Resultado del receptor

```
RECEPTOR - PRUEBA 1
Entrada: HAMMING|0110011|4|0|BINARIO
Algoritmo recibido: HAMMING
Trama recibida: 0110011
Longitud original: 4
Padding: 0
Tipo: BINARIO
¿Se detectaron errores?: No
Síndrome: 0
Posición del error: No aplica
Trama corregida: No aplica
Estado: TRAMA VÁLIDA
Mensaje binario recuperado: 1011
```


Resultado observado: El síndrome fue 0 y el receptor recuperó correctamente 1011.

Salida del emisor

```
PS C:\Users\ninan\OneDrive\Documentos\Escritorio\UVG\VIII SEMESTRE\Redes\Redes> & C:\Users\ninan\OneDrive\Documentos\Escritorio\UVG\VIII SEMESTRE\Redes\Redes\emisor.py"
=====
EMISOR - DETECCIÓN Y CORRECCIÓN DE ERRORES
=====

Tipo de mensaje:
1. Cadena binaria
2. Texto ASCII
Seleccione una opción: 1
Ingrese el mensaje binario: 11010110

Algoritmo:
1. Hamming, corrección de un error
2. Fletcher checksum, detección de errores
Seleccione una opción: 1

Bits de paridad agregados: 4

Aplicación de ruido:
1. No aplicar ruido y modificar los bits manualmente
2. Aplicar ruido automáticamente según una probabilidad
Seleccione una opción: 1

=====
RESULTADO DEL EMISOR
=====

Mensaje original: 11010110
Datos binarios originales: 11010110
Tipo de mensaje: BINARIO
Algoritmo: HAMMING
Longitud original en bits: 8
Padding agregado: 0
Trama antes del ruido: 001010100110
Probabilidad utilizada: 0.0
Trama que se enviará: 001010100110
Bits modificados por el ruido: ninguno

COPIAR Y ENVIAR AL RECEPTOR:
HAMMING|001010100110|8|0|BINARIO
=====
PS C:\Users\ninan\OneDrive\Documentos\Escritorio\UVG\VIII SEMESTRE\Redes\Redes> |
```

Resultado del receptor

```
RECEPTOR - PRUEBA 2
Entrada: HAMMING|001010100110|8|0|BINARIO
Algoritmo recibido: HAMMING
Trama recibida: 001010100110
Longitud original: 8
Padding: 0
Tipo: BINARIO
¿Se detectaron errores?: No
Síndrome: 0
Posición del error: No aplica
Trama corregida: No aplica
Estado: TRAMA VÁLIDA
Mensaje binario recuperado: 11010110
```

Resultado observado: El receptor indicó que la trama era válida y recuperó los ocho bits originales.

Salida del emisor

```
PS C:\Users\ninan\OneDrive\Documentos\Escritorio\UVG\VIII SEMESTRE\Redes\Redes> & C:\Users\ninan\OneDrive\Documentos\Escritorio\UVG\VIII SEMESTRE\Redes\Redes\emisor.py"
=====
EMISOR - DETECCIÓN Y CORRECCIÓN DE ERRORES
=====

Tipo de mensaje:
1. Cadena binaria
2. Texto ASCII
Seleccione una opción: 2
Ingrese el texto ASCII: Hola

Algoritmo:
1. Hamming, corrección de un error
2. Fletcher checksum, detección de errores
Seleccione una opción: 1

Bits de paridad agregados: 6

Aplicación de ruido:
1. No aplicar ruido y modificar los bits manualmente
2. Aplicar ruido automáticamente según una probabilidad
Seleccione una opción: 1

=====
RESULTADO DEL EMISOR
=====

Mensaje original: Hola
Datos binarios originales: 01001000011011110110110001100001
Tipo de mensaje: ASCII
Algoritmo: HAMMING
Longitud original en bits: 32
Padding agregado: 0
Trama antes del ruido: 11001001100001110111101101100010100001
Probabilidad utilizada: 0.0
Trama que se enviará: 11001001100001110111101101100010100001
Bits modificados por el ruido: ninguno

COPIAR Y ENVIAR AL RECEPTOR:
HAMMING|11001001100001110111101101100010100001|32|0|ASCII
=====
PS C:\Users\ninan\OneDrive\Documentos\Escritorio\UVG\VIII SEMESTRE\Redes\Redes> |
```

Resultado del receptor

```
RECEPTOR - PRUEBA 3
Entrada: HAMMING|11001001100001110111101101100010100001|32|0|ASCII
Algoritmo recibido: HAMMING
Trama recibida: 11001001100001110111101101100010100001
Longitud original: 32
Padding: 0
Tipo: ASCII
¿Se detectaron errores?: No
Síndrome: 0
Posición del error: No aplica
Trama corregida: No aplica
Estado: TRAMA VÁLIDA
Mensaje binario recuperado: 01001000011011110110110001100001
Mensaje ASCII recuperado: Hola
```

Resultado observado: El receptor recuperó los 32 bits originales y los convirtió correctamente al texto Hola.

Salida del emisor

```
=====
Tipo de mensaje:
1. Cadena binaria
2. Texto ASCII
Seleccione una opción: 1
Ingrese el mensaje binario: 1011

Algoritmo:
1. Hamming, corrección de un error
2. Fletcher checksum, detección de errores
Seleccione una opción: 2

Variante de Fletcher:
1. Fletcher-8
2. Fletcher-16
3. Fletcher-32
Seleccione una opción: 2

Aplicación de ruido:
1. No aplicar ruido y modificar los bits manualmente
2. Aplicar ruido automáticamente según una probabilidad
Seleccione una opción: 1
```

=====

RESULTADO DEL EMISOR

=====

```
Mensaje original: 1011
Datos binarios originales: 1011
Tipo de mensaje: BINARIO
Algoritmo: FLETCHER16
Longitud original en bits: 4
Padding agregado: 4
Checksum: 1011000010110000
Trama antes del ruido: 101100001011000010110000
Probabilidad utilizada: 0.0
Trama que se enviará: 101100001011000010110000
Bits modificados por el ruido: ninguno
```

```
COPIAR Y ENVIAR AL RECEPTOR:
FLETCHER16|101100001011000010110000|4|4|BINARIO
```

```
=====
PS C:\Users\ninan\OneDrive\Documentos\Escritorio\UVG\VIII SEMESTRE\Redes\Redes> |
```

🔗 Ninaswiftie09 (8 minutes ago) Ln 41

Resultado del receptor

```
RECEPTOR FLETCHER - PRUEBA 4
Entrada: FLETCHER16|101100001011000010110000|4|4|BINARIO
Algoritmo recibido: FLETCHER16
Trama recibida: 101100001011000010110000
Longitud original: 4
Padding: 4
Tipo: BINARIO
Checksum recibido: 1011000010110000
Checksum calculado: 1011000010110000
¿Se detectaron errores?: No
Estado: TRAMA VÁLIDA
Mensaje binario recuperado: 1011
```

Resultado observado: El checksum recibido coincidió con el calculado. Después de eliminar el padding, se recuperó 1011.

Salida del emisor

```
=====
Tipo de mensaje:
1. Cadena binaria
2. Texto ASCII
Seleccione una opción: 1
Ingrese el mensaje binario: 11010110

Algoritmo:
1. Hamming, corrección de un error
2. Fletcher checksum, detección de errores
Seleccione una opción: 2

Variante de Fletcher:
1. Fletcher-8
2. Fletcher-16
3. Fletcher-32
Seleccione una opción: 2

Aplicación de ruido:
1. No aplicar ruido y modificar los bits manualmente
2. Aplicar ruido automáticamente según una probabilidad
Seleccione una opción: 1
```

=====

RESULTADO DEL EMISOR

=====

```
Mensaje original: 11010110
Datos binarios originales: 11010110
Tipo de mensaje: BINARIO
Algoritmo: FLETCHER16
Longitud original en bits: 8
Padding agregado: 0
Checksum: 1101011011010110
Trama antes del ruido: 1101011011010110110110
Probabilidad utilizada: 0.0
Trama que se enviará: 1101011011010110110110
Bits modificados por el ruido: ninguno
```

```
COPIAR Y ENVIAR AL RECEPTOR:
FLETCHER16|1101011011010110110110|8|0|BINARIO
```

```
=====
PS C:\Users\ninan\OneDrive\Documentos\Escritorio\UVG\VIII SEMESTRE\Redes\Redes>
```

Resultado del receptor

```
RECEPTOR FLETCHER - PRUEBA 5
Entrada: FLETCHER16|1101011011010110110110|8|0|BINARIO
Algoritmo recibido: FLETCHER16
Trama recibida: 1101011011010110110110
Longitud original: 8
Padding: 0
Tipo: BINARIO
Checksum recibido: 1101011011010110
Checksum calculado: 1101011011010110
¿Se detectaron errores?: No
Estado: TRAMA VÁLIDA
Mensaje binario recuperado: 11010110
```


Resultado observado: Los checksums coincidieron y la trama fue aceptada.

Salida del emisor

```
=====
Tipo de mensaje:
1. Cadena binaria
2. Texto ASCII
Seleccione una opción: 2
Ingrese el texto ASCII: Hola

Algoritmo:
1. Hamming, corrección de un error
2. Fletcher checksum, detección de errores
Seleccione una opción: 2

Variante de Fletcher:
1. Fletcher-8
2. Fletcher-16
3. Fletcher-32
Seleccione una opción: 2

Aplicación de ruido:
1. No aplicar ruido y modificar los bits manualmente
2. Aplicar ruido automáticamente según una probabilidad
Seleccione una opción: 1

=====
RESULTADO DEL EMISOR
=====
Mensaje original: Hola
Datos binarios originales: 01001000011011110110110001100001
Tipo de mensaje: ASCII
Algoritmo: FLETCHER16
Longitud original en bits: 32
Padding agregado: 0
Checksum: 1010100110000101
Trama antes del ruido: 010010000110111101101100011000011010100110000101
Probabilidad utilizada: 0.0
Trama que se enviará: 010010000110111101101100011000011010100110000101
Bits modificados por el ruido: ninguno

COPIAR Y ENVIAR AL RECEPTOR:
FLETCHER16|010010000110111101101100011000011010100110000101|32|0|ASCII
=====
PS C:\Users\ninan\OneDrive\Documentos\Escritorio\UVG\VIII SEMESTRE\Redes\Redes>
```

Resultado del receptor

```
RECEPTOR FLETCHER - PRUEBA 6
Entrada: FLETCHER16|010010000110111101101100011000011010100110000101|32|0|ASCII
Algoritmo recibido: FLETCHER16
Trama recibida: 010010000110111101101100011000011010100110000101
Longitud original: 32
Padding: 0
Tipo: ASCII
Checksum recibido: 1010100110000101
Checksum calculado: 1010100110000101
¿Se detectaron errores?: No
Estado: TRAMA VÁLIDA
Mensaje binario recuperado: 01001000011011110110110001100001
Mensaje ASCII recuperado: Hola
```

Resultado observado: Los checksums coincidieron y el receptor recuperó correctamente Hola.

6.2. Pruebas con un error

En las pruebas 7 a 12 se modificó un bit dentro del campo que contiene la trama.

```
RECEPTOR - PRUEBA 7
Entrada: HAMMING|0111011|4|0|BINARIO
Algoritmo recibido: HAMMING
Trama recibida: 0111011
Longitud original: 4
Padding: 0
Tipo: BINARIO
¿Se detectaron errores?: Sí
Síndrome: 4
Posición del error: 4
Trama corregida: 0110011
Estado: ERROR CORREGIDO
Mensaje binario recuperado: 1011
```

Figura 7: Prueba 7: Hamming con el mensaje 1011 y un error

Resultado observado: El síndrome fue 4. El receptor corrigió el bit de la posición 4 y recuperó correctamente 1011.

```
RECEPTOR - PRUEBA 8
Entrada: HAMMING|001110100110|8|0|BINARIO
Algoritmo recibido: HAMMING
Trama recibida: 001110100110
Longitud original: 8
Padding: 0
Tipo: BINARIO
¿Se detectaron errores?: Sí
Síndrome: 4
Posición del error: 4
Trama corregida: 001010100110
Estado: ERROR CORREGIDO
Mensaje binario recuperado: 11010110
```

Figura 8: Prueba 8: Hamming con el mensaje 11010110 y un error

Resultado observado: El receptor encontró el error en la posición 4, corrigió la trama y recuperó el mensaje original.

```

RECEPTOR - PRUEBA 9
Entrada: HAMMING|11011001100001110111101101100010100001|32|0|ASCII
Algoritmo recibido: HAMMING
Trama recibida: 11011001100001110111101101100010100001
Longitud original: 32
Padding: 0
Tipo: ASCII
¿Se detectaron errores?: Sí
Síndrome: 4
Posición del error: 4
Trama corregida: 11011001100001110111101101100010100001
Estado: ERROR CORREGIDO
Mensaje binario recuperado: 01001000011011110110110001100001
Mensaje ASCII recuperado: Hola

```

Figura 9: Prueba 9: Hamming con el texto Hola y un error

Resultado observado: El error fue corregido y el texto ASCII recuperado fue Hola.

```

RECEPTOR FLETCHER - PRUEBA 10
Entrada: FLETCHER16|001100001011000010110000|4|4|BINARIO
Algoritmo recibido: FLETCHER16
Trama recibida: 001100001011000010110000
Longitud original: 4
Padding: 4
Tipo: BINARIO
Checksum recibido: 1011000010110000
Checksum calculado: 0011000000110000
¿Se detectaron errores?: Sí
Estado: ERROR DETECTADO
Mensaje binario recuperado: No disponible
Posición del error: No aplica
Trama corregida: No aplica
Acción: trama descartada; Fletcher no corrige errores.

```

Figura 10: Prueba 10: Fletcher-16 con el mensaje 1011 y un error

Resultado observado: El checksum calculado fue diferente al recibido. Fletcher detectó el error y descartó la trama.

```

RECEPTOR FLETCHER - PRUEBA 11
Entrada: FLETCHER16|010101101101101011011010110110|8|0|BINARIO
Algoritmo recibido: FLETCHER16
Trama recibida: 010101101101011011010110
Longitud original: 8
Padding: 0
Tipo: BINARIO
Checksum recibido: 1101011011010110
Checksum calculado: 0101011001010110
¿Se detectaron errores?: Sí
Estado: ERROR DETECTADO
Mensaje binario recuperado: No disponible
Posición del error: No aplica
Trama corregida: No aplica
Acción: trama descartada; Fletcher no corrige errores.

```

Figura 11: Prueba 11: Fletcher-16 con el mensaje 11010110 y un error

Resultado observado: El receptor detectó la modificación y no intentó recuperar el mensaje.

```

RECEPTOR FLETCHER - PRUEBA 12
Entrada: FLETCHER16|110010000110111101101100011000011010100110000101|32|0|ASCII
Algoritmo recibido: FLETCHER16
Trama recibida: 110010000110111101101100011000011010100110000101
Longitud original: 32
Padding: 0
Tipo: ASCII
Checksum recibido: 1010100110000101
Checksum calculado: 1010101100000110
¿Se detectaron errores?: Sí
Estado: ERROR DETECTADO
Mensaje binario recuperado: No disponible
Posición del error: No aplica
Trama corregida: No aplica
Acción: trama descartada; Fletcher no corrige errores.

```

Figura 12: Prueba 12: Fletcher-16 con el texto Hola y un error

Resultado observado: El checksum permitió detectar el error y la trama fue descartada.

6.3. Pruebas con dos errores

En las pruebas 13 a 18 se modificaron dos bits de cada trama.

```
RECEPTOR - PRUEBA 13
Entrada: HAMMING|0011011|4|0|BINARIO
Algoritmo recibido: HAMMING
Trama recibida: 0011011
Longitud original: 4
Padding: 0
Tipo: BINARIO
¿Se detectaron errores?: Sí
Síndrome: 6
Posición del error: 6
Trama corregida: 0011001
Estado: ERROR CORREGIDO
Mensaje binario recuperado: 1001
Nota: con múltiples errores Hamming sin paridad global no es confiable.
```

Figura 13: Prueba 13: Hamming con el mensaje 1011 y dos errores

Resultado observado: El síndrome indicó la posición 6 y el receptor realizó una corrección. Sin embargo, recuperó 1001 en lugar de 1011.

```
RECEPTOR - PRUEBA 14
Entrada: HAMMING|011110100110|8|0|BINARIO
Algoritmo recibido: HAMMING
Trama recibida: 011110100110
Longitud original: 8
Padding: 0
Tipo: BINARIO
¿Se detectaron errores?: Sí
Síndrome: 6
Posición del error: 6
Trama corregida: 011111100110
Estado: ERROR CORREGIDO
Mensaje binario recuperado: 11110110
Nota: con múltiples errores Hamming sin paridad global no es confiable.
```

Figura 14: Prueba 14: Hamming con el mensaje 11010110 y dos errores

Resultado observado: El receptor mostró un error corregido, pero recuperó 11110110. Esto demuestra que el resultado no fue confiable.

```

RECEPTOR - PRUEBA 15
Entrada: HAMMING|10011001100001110111101101100010100001|32|0|ASCII
Algoritmo recibido: HAMMING
Trama recibida: 10011001100001110111101101100010100001
Longitud original: 32
Padding: 0
Tipo: ASCII
¿Se detectaron errores?: Sí
Síndrome: 6
Posición del error: 6
Trama corregida: 10011101100001110111101101100010100001
Estado: ERROR CORREGIDO
Mensaje binario recuperado: 01101000011011110110110001100001
Mensaje ASCII recuperado: hola
Nota: con múltiples errores Hamming sin paridad global no es confiable.

```

Figura 15: Prueba 15: Hamming con el texto Hola y dos errores

Resultado observado: El receptor recuperó hola, con la primera letra en minúscula. Aunque indicó que el error fue corregido, el mensaje no fue igual al original.

```

RECEPTOR FLETCHER - PRUEBA 16
Entrada: FLETCHER16|011100001011000010110000|4|4|BINARIO
Algoritmo recibido: FLETCHER16
Trama recibida: 011100001011000010110000
Longitud original: 4
Padding: 4
Tipo: BINARIO
Checksum recibido: 1011000010110000
Checksum calculado: 0111000001110000
¿Se detectaron errores?: Sí
Estado: ERROR DETECTADO
Mensaje binario recuperado: No disponible
Posición del error: No aplica
Trama corregida: No aplica
Acción: trama descartada; Fletcher no corrige errores.

```

Figura 16: Prueba 16: Fletcher-16 con el mensaje 1011 y dos errores

Resultado observado: El checksum calculado fue diferente al recibido y la trama fue descartada.


```

RECEPTOR FLETCHER - PRUEBA 17
Entrada: FLETCHER16|00010110110110110110110|8|0|BINARIO
Algoritmo recibido: FLETCHER16
Trama recibida: 00010110110110110110
Longitud original: 8
Padding: 0
Tipo: BINARIO
Checksum recibido: 11010110110110
Checksum calculado: 0001011000010110
¿Se detectaron errores?: Sí
Estado: ERROR DETECTADO
Mensaje binario recuperado: No disponible
Posición del error: No aplica
Trama corregida: No aplica
Acción: trama descartada; Fletcher no corrige errores.

```

Figura 17: Prueba 17: Fletcher-16 con el mensaje 11010110 y dos errores

Resultado observado: Fletcher detectó las modificaciones y evitó aceptar información incorrecta.

```

RECEPTOR FLETCHER - PRUEBA 18
Entrada: FLETCHER16|100010000110111101101100011000110100110000101|32|0|ASCII
Algoritmo recibido: FLETCHER16
Trama recibida: 100010000110111101101100011000110100110000101
Longitud original: 32
Padding: 0
Tipo: ASCII
Checksum recibido: 1010100110000101
Checksum calculado: 101010111000101
¿Se detectaron errores?: Sí
Estado: ERROR DETECTADO
Mensaje binario recuperado: No disponible
Posición del error: No aplica
Trama corregida: No aplica
Acción: trama descartada; Fletcher no corrige errores.

```

Figura 18: Prueba 18: Fletcher-16 con el texto Hola y dos errores

Resultado observado: El receptor detectó los errores y descartó la trama sin intentar corregirla.

7. Análisis gráfico

7.1. Cantidad de información adicional

Para Hamming, la información adicional corresponde a los bits de paridad. Para Fletcher-16 se incluyen los 16 bits del checksum y el padding necesario.

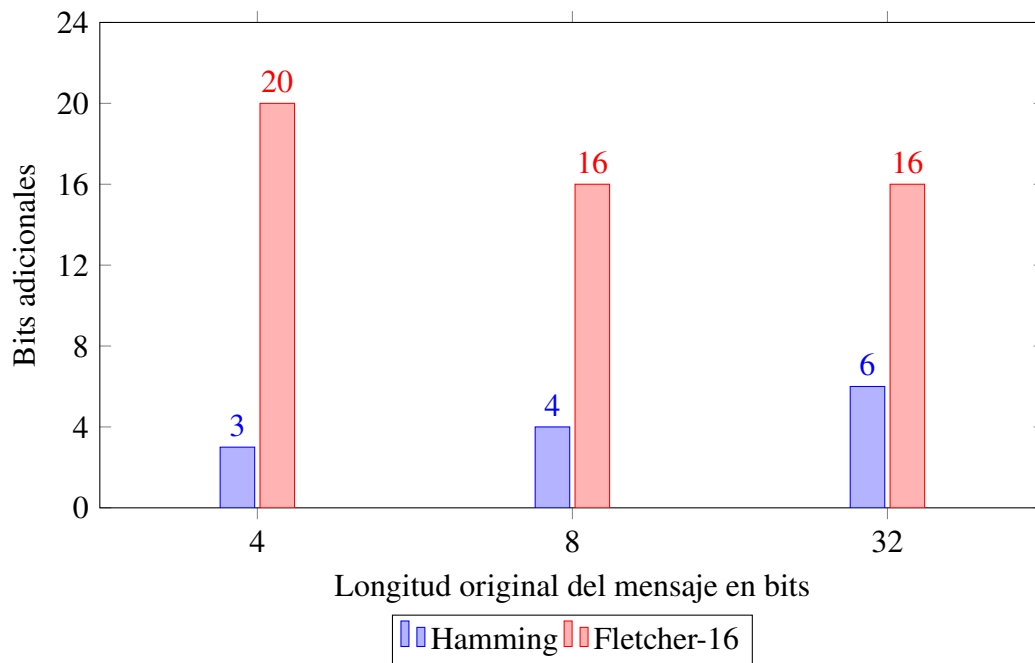


Figura 19: Comparación del overhead de los algoritmos.

Hamming utilizó menos bits adicionales en los tres tamaños probados. En el mensaje de cuatro bits, Fletcher necesitó cuatro bits de padding y 16 bits de checksum, por lo que su overhead fue mayor.

7.2. Resultados correctos según la función del algoritmo

La siguiente gráfica muestra cuántas de las tres pruebas de cada escenario produjeron el comportamiento correcto para la función del algoritmo.

En Hamming, se consideró correcto recuperar el mensaje original. En Fletcher, se consideró correcto aceptar una trama intacta o detectar y descartar una trama alterada.

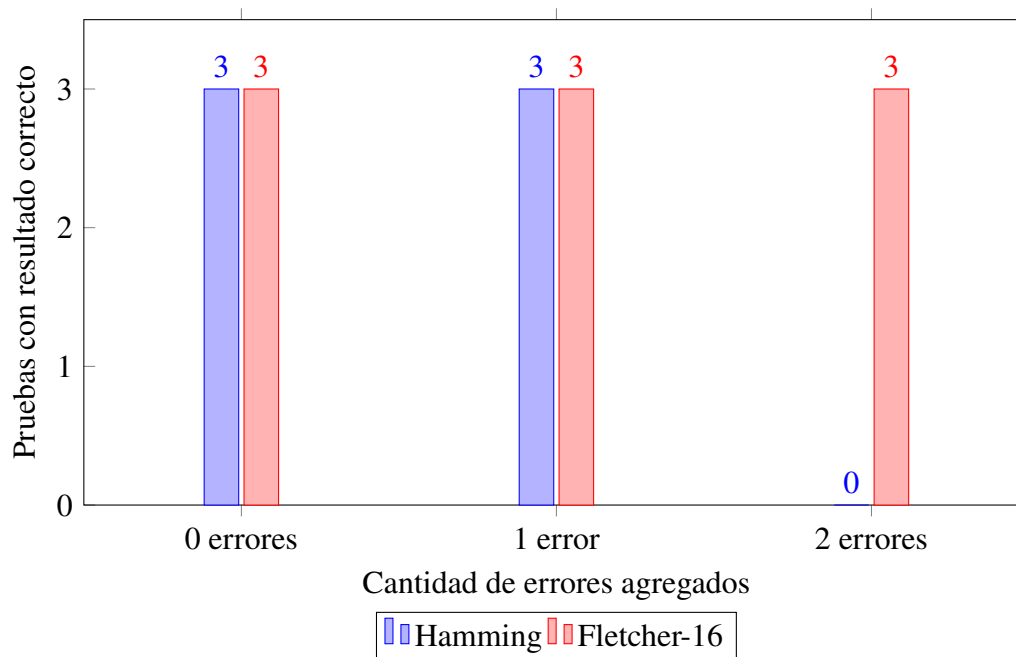


Figura 20: Resultados obtenidos de acuerdo con el propósito de cada algoritmo.

8. Discusión

8.1. Funcionamiento de Hamming

Hamming presentó un buen funcionamiento cuando la trama estaba intacta y cuando se modificó un solo bit. En las pruebas 7, 8 y 9, el síndrome indicó correctamente la posición 4. Después de invertir ese bit, el receptor recuperó los tres mensajes originales.

El comportamiento cambió al introducir dos errores. En las pruebas 13, 14 y 15, el receptor calculó un síndrome válido y realizó una corrección, pero el resultado no coincidió con la información original.

Este fue el caso más cercano a “engañar” al algoritmo. El receptor indicó que el error había sido corregido, aunque los mensajes obtenidos fueron 1001, 11110110 y hola. Esto ocurre porque el Hamming utilizado está diseñado para corregir un solo error y no posee una paridad global que permita distinguir algunos errores múltiples.

8.2. Funcionamiento de Fletcher-16

Fletcher-16 aceptó correctamente las tres tramas sin errores. Cuando se modificó uno o dos bits, el checksum calculado fue diferente al checksum recibido.

En las seis pruebas con alteraciones, Fletcher detectó los errores y descartó las tramas. No pudo recuperar los mensajes porque su función es únicamente detectar cambios.

Dentro de las pruebas realizadas no se encontró una combinación que produjera el mismo

checksum después de modificar los bits. Sin embargo, un checksum tiene una cantidad limitada de valores, por lo que no garantiza que todas las alteraciones posibles sean detectadas.

8.3. Comparación

No existe un algoritmo que sea mejor en todas las situaciones.

Hamming fue más útil cuando ocurrió un solo error, ya que permitió recuperar el mensaje sin solicitar una nueva transmisión. También agregó menos bits de redundancia que Fletcher-16 en los tamaños probados.

Fletcher-16 fue más confiable para detectar que una trama con varios errores había sido alterada. Aunque no corrigió la información, evitó que el receptor aceptara mensajes incorrectos.

Un algoritmo de detección resulta conveniente cuando es posible pedir que la información sea enviada nuevamente. Un algoritmo de corrección resulta útil cuando una retransmisión es costosa o no está disponible, siempre que la cantidad de errores se encuentre dentro de su capacidad.

9. Comentario grupal

La práctica permitió observar de forma directa la importancia de los bits de redundancia. Aunque estos bits aumentan el tamaño de la trama, permiten identificar si la información fue modificada durante la transmisión.

También se comprobó que detectar y corregir no significan lo mismo. Fletcher pudo reconocer los cambios, pero no repararlos. Hamming pudo reparar un error, pero produjo resultados incorrectos cuando se modificaron varios bits.

El uso de dos lenguajes diferentes también permitió comprobar que la comunicación no depende del lenguaje utilizado, siempre que ambos programas respeten el mismo formato y las mismas reglas.

10. Conclusiones

1. El código de Hamming recuperó correctamente los mensajes cuando ocurrió un solo error.
2. Hamming sin paridad global no fue confiable cuando se modificaron dos bits, ya que realizó correcciones incorrectas.
3. Fletcher-16 detectó correctamente todos los errores de uno y dos bits utilizados en las pruebas.
4. Fletcher-16 no puede corregir los errores, por lo que las tramas alteradas deben descartarse o enviarse nuevamente.

5. Hamming agregó menos información adicional que Fletcher-16 para los tamaños de mensaje evaluados.
6. El formato compartido permitió que el emisor en Python y el receptor en Java procesaran la misma información correctamente.

11. Repositorio

El código fuente del emisor, el receptor, las pruebas y las capturas se encuentra en la rama Laboratorio2 del siguiente repositorio:

`github.com/Ninaswiftie09/Redes/tree/Laboratorio2`

Referencias

- [1] Universidad del Valle de Guatemala. *Laboratorio 2: Esquemas de detección y corrección de errores*. Curso CC3067 Redes, 2026.
- [2] Universidad del Valle de Guatemala. *Guía resumida paso a paso para el Laboratorio 2: Esquemas de detección y corrección de errores*. 2026.